Summary of CPEN 411

Tom Wang

Fall, 2023

1 Terminology

The Iron law

Sum of each cycle execution time is the total time.

CPI = Cycle per Instruction

$$\frac{\text{CPU time}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock Cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Clock Cycle}}$$

$$\text{Execution Time} = \frac{1}{\text{Performance}} = (\text{Instr. Count}) \times (\text{CPI}) \times (\text{cycle time})$$

$$\text{speedup} = \frac{\text{Q1}}{\text{Q2}}$$

The speedup of Q2 respect to Q1 is xxx.

Amdahl's Law

$$\text{ExTime}_{new} = \text{ExTime}_{old} \times \left[(1 - \text{Fraction}_{enhanced}) + \frac{\text{Fraction}_{enhanced}}{\text{Speedup}_{enhanced}} \right]$$

$$\text{Speedup}_{overall} = \frac{\text{ExTime}_{old}}{\text{ExTime}_{new}} = \frac{1}{(1 - \text{Fraction}_{enhanced}) + \frac{\text{Fraction}_{enhanced}}{\text{Speedup}_{enhanced}}}$$

Dynamic Power

Foe CMOS chips, traditional dominant energy consumption has been in switching transsitors, called **dynamic power**.

$$\text{Power}_{\text{dynamic}} = 0.5 * \text{CapacitiveLoad} * \text{Voltage}^2 * \text{FrequencySwitched}$$

For mobile devices, energy might be a better metric:

$$\text{Energy}_{\text{dynamic}} = \text{CapacitiveLoad} * \text{Voltage}^2$$

Capacitive load is a function of number of transistors connected to ouput and technology, which determines capacitance of wires and transistors.

Dropping voltage helps both power and energy.

For a fixed task, slowing clock rate (frequency switched) reduced pwoer, but not the total energy. However, slower clock rate migh allow lower voltage.

Clock gating: Turn off clocks for the parts that we do not use which saves energy and dynamic power. However, clock skew rises up, it is hard to be precise.

Stactic power

Leakage current flows even when a transistor is off. So there is static power.

$$\text{Power}_{\text{static}} = \text{Current}_{\text{static}} * \text{Voltage}$$

Leakage of current increases in processors with smaller transsitor sizes.

Increasing the number of transistors increases power.

Very slow power systems can gate voltage to inactive modules.

2 Memory technologies

Learning objectives

- Identify memory technologies used today
- Contrast SRAM, RF, CAM, and DRAM structures
- Describe the tradeoffs (density, latency, power)

Memory hirerarchy

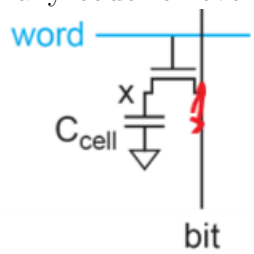
First of all, some definations:

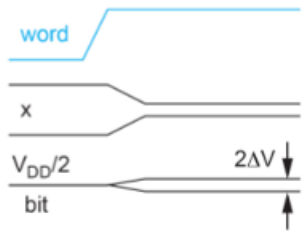
Storage: SSD or HDD

DARM: random-access semiconductor memory. Usually consisting of a tiny **capacitor** and a **transistor**. DRAM requires an external memory refresh because of slow leakage on capacitor.

One trick to not block any code: rewrite a certain portion of it at once, split it averagely into the refresh cycle. This way DRAM does not block any code for every refresh cycle.

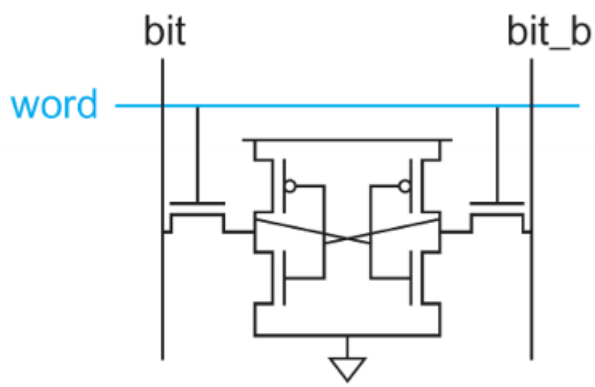
- 1T + 1C: cheap as chips!
- read operation
  - precharge bitline to ½ V_{DD}
  - cap changes bitline V a little bit
  - read **destructive**: write value back



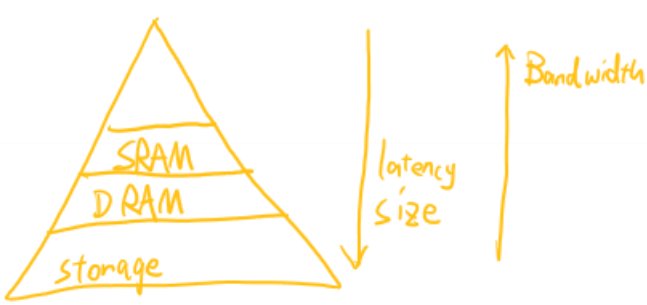


SRAM: A type of random access memory. Unlike DRAM, it does not need referesh. SRAM is commonly used for computer's cache memory, such as L2 or L3 cache. SRAM contains 6 transistors in total if counting the transistors to the bit line. Otherwise, 4 transistors.

- reading:
  - precharge both bitlines **high**
  - cell will pull down one of the bitlines
- writing:
  - one bitline pulls cell down



The following is the hierarchy.

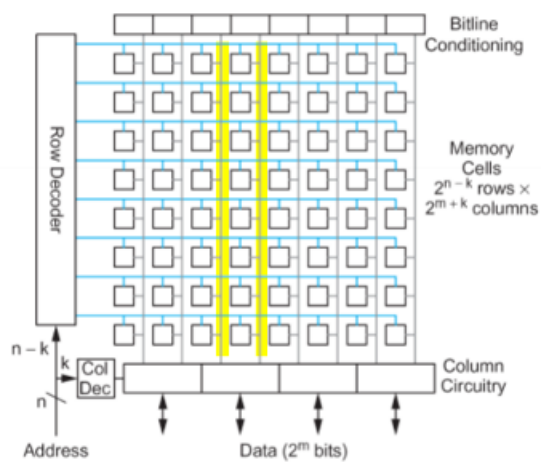


Memory addressing patterns

access pattern	**random access** address -> data	**associative lookup** key -> has match?
example application	register file direct-mapped caches main memory	associative caches / TLBs reservation stations load-store queue
technology	SRAM, RF, DRAM	Content Addressible Memory CAM

RAM addressing

- cells store data
- addr[$n-1$:k] selects row
- addr[$k-1$:0] selects word



We turn on the whole role and the whole word to select the address. Here we want 'n' to be close to 'm'. If n>>m, n will dominate latency. Vice versa.

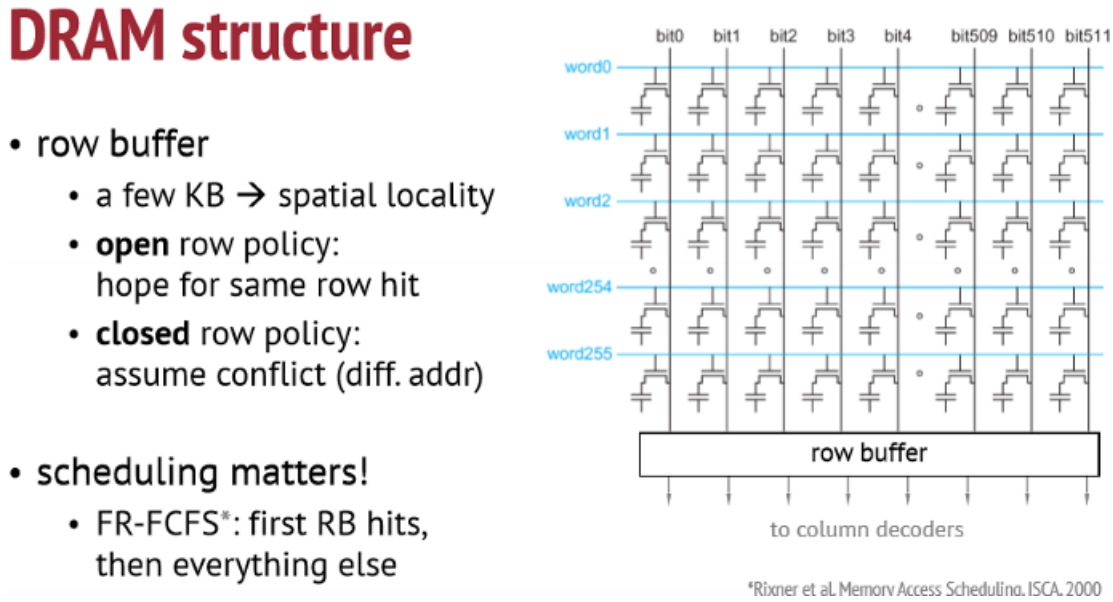
To read, address drive the word line, cells drive the bitlines. Then bitlines read out and amplified.

To write, address drives the word line. The value drives the bitlines. Then we write the bitline values to the cells. Now cells keep the data.

For DRAM, each bank has a single row buffer. It act as a cache within DRAM. Row buffer can serve mutiple instructions. The first one takes much longer since it needs to fetch from the memory. The rest can directly read from the buffer, much faster.

- If an access stream has locality, a row buffer is kept open. (open policy) Row buffer hits are cheap. Row buffer miss is a bank conflict and expensive because precharge is on the critical path

- If an access stream has little locality, bitlines are precharged immediately after access (close-page policy) Nearly every access is a row buffer miss. The precharge is usually not on the critical path.



Rowhammer

Repeated same row hits in recent DRAMs can flip bits in neighboring row!

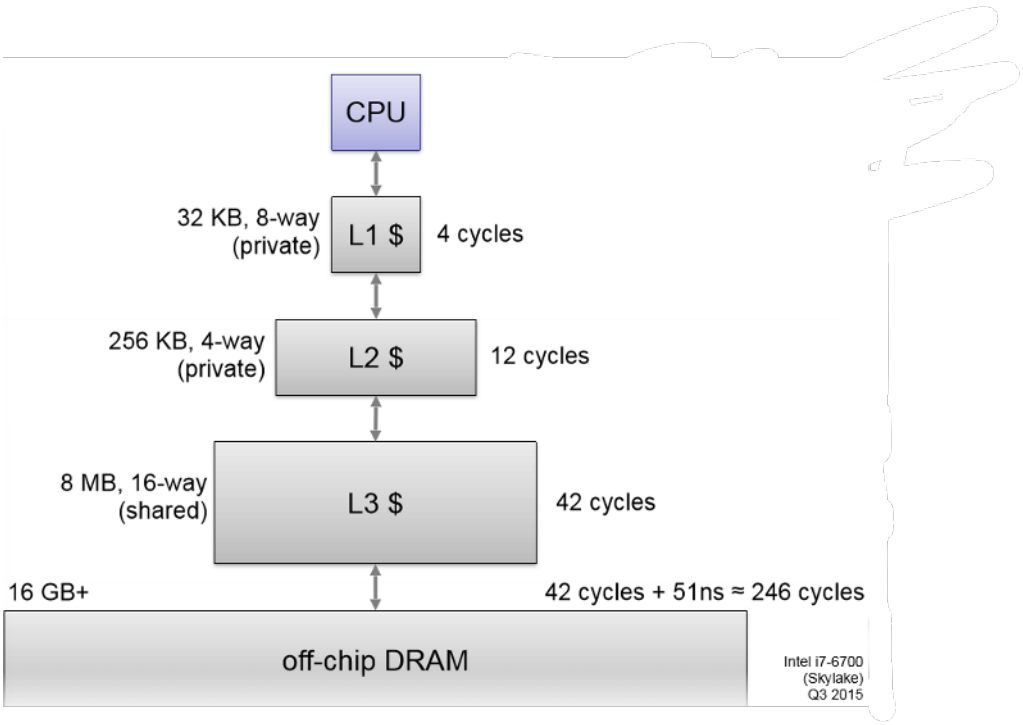
However, you need to access both the upper and lower address of the target row. Since if you only access one of them, then the row buffer will take all the calls due to optimization.

3 Caches and Prefetching

Avaliable technologies

- DRAM: dense and cheap but slow (10s of ns)
- SRAM: about 5 times more area per bit but fast! For SRAMs: samller the size, faster the response.
- So now, we have a tradeoff between fast and big.

To solve the tradeoff problems, we establish a leveled memory system:



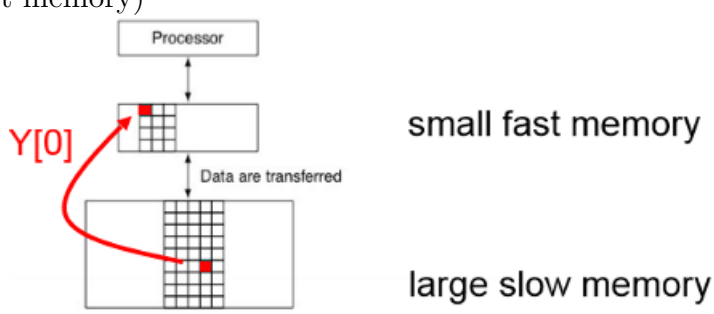
Memory access locality

First some terminology:

Temporal locality: If the program accesses a single location, then accesses it soon thereafter.

For eaxmple, you read the location and write to that location afterwards. Y=Y+5

Idea of temporal locality is to keep copy of recently accessed data nearby. (i.e., in small, fast memory)



Here we move the data from DRAM into cache for faster multiple visits.

There are two ways to manage fast storage:

- Option 1: scractchpad:
  - program copies most useful data. Unlike cache, srachpad memory is typically not managed by software.

  - programmer or compiler decides what data or instructions should be placed in the scratchpad
  - used in GPUs and other contexts
  - The capacity of scratchpad memory is limited and is typically smaller than the cache memory but larger than CPU registers.
  - scractchpad memory is usually a signle contiguous block of memory, and its size and organiztion are determined by the programmer.

- Option 2: cache
  - Managied automatically in hardware where it tracks locality patterns.
  - Functionally transparent to the programmer(Software).
  - Cache is organized into multiple levels, typically L1, L2 and sometimes L3.

Fully associated cache:

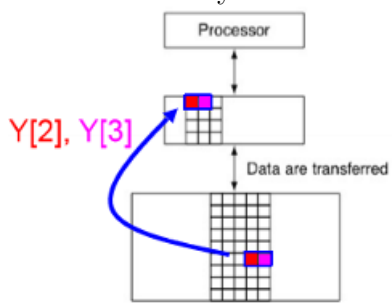
- Allows any block of data from the main memory to be placed in any cache location.
- There are no restrictions on where a particular block of data can be stored.
- There is no set index to determine where to place or find data. We use tags to keep track of which block of data is stored in each cache location:
  - Memory block address: to identyfy which memory block is stored in the cache location
  - Valid Bit: A valid bit indicates whether the cache location contains valid data. If the valid bit is set, it means that the data in that location is valid and corresponds to the given tag.
  - Dirty/Modified Bit: To indicate whether the data in the cache location has been modified and needs to be written back to the main memory.
- Fully associative caches tend to have fewer cache misses due to conflicts (where multiple memory blocks complete for the same cache location) because they can accommodate any data block in any cache location.
- Implementing a fully associative cache can be more expensive in terms of harware complexity and cost compared to others. The need for tags and additional logic for managing cache entries adds to the hardware overhead.
- Fully associative caches can achieve high hit rates for data access because they can match requested data with any cache entry, regardless of its location within the cache. This makes them particularly effective for applications with irregular memory access patterns.

Problems with fully associated caches:

- Tag must store full address of the memory
- Address can be in any row, have to search all rows.

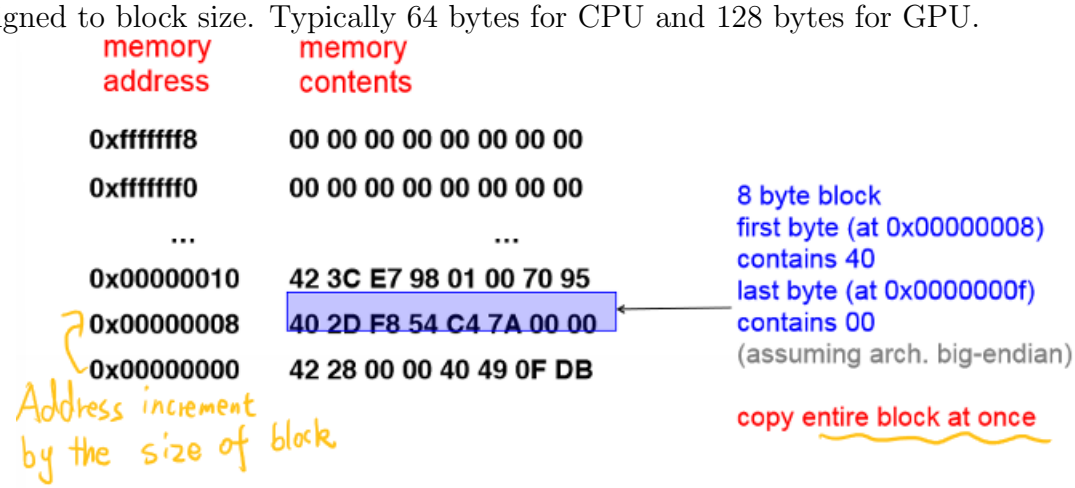
Spatial locality: After accessing one location, program accesses nearby location.

For eaxmple, Foreach (var i in Array). accesses all components of array starting the address of the array.

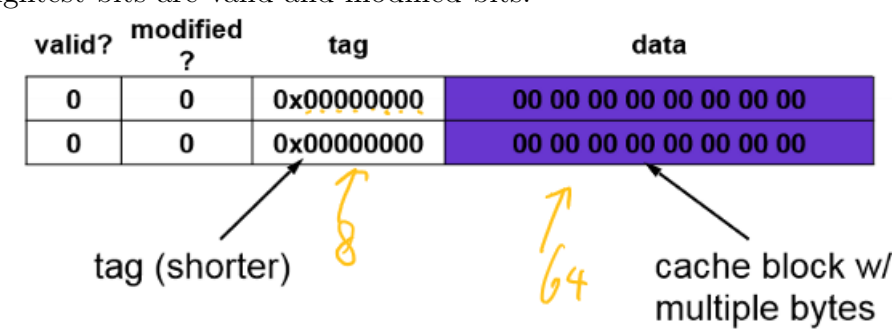


IDEA: when filling cache entry, also transfer adjacent data.

Memory divide into blocks (cachelines) with power-of-two size. Blocks start at addresses aligned to block size. Typically 64 bytes for CPU and 128 bytes for GPU.



So we include the 8 byte address in tag, 64 bytes data in lower bits of the block. The hightest bits are valid and modified bits.



Block offset

Block offset is a component of memory addresses that helps determin the position of a specific byte within a memory block or cache line.

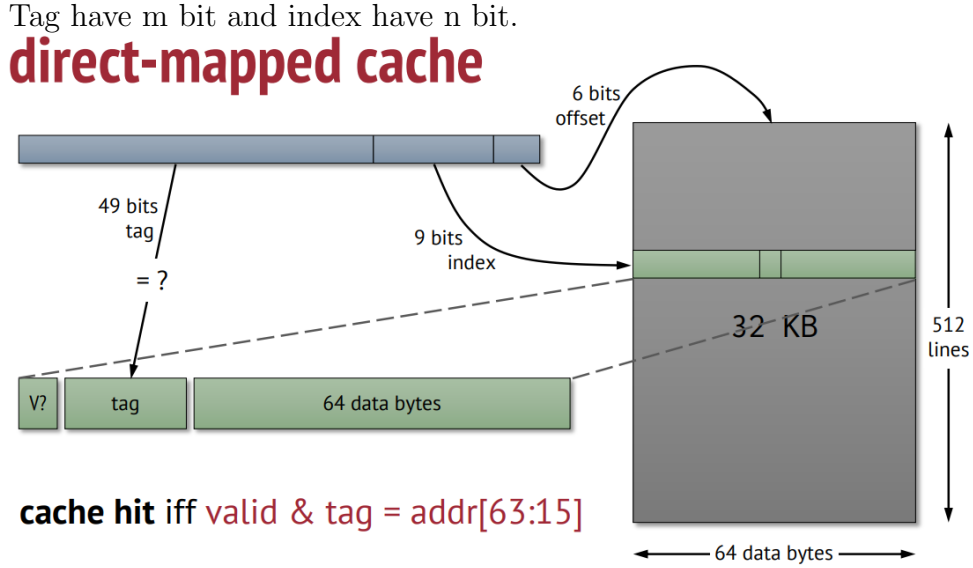
To address individual bytes within a block, the block offset is represented in binary format. The number of bits required for the block offset depends on the block size. For example, if the block size is 64 bytes, you would need 6 bits to uniquely address any byte within that block.

$$\text{Block Offset Bits} = \log_2(\text{Block Size in Bytes})$$

Direct Mapped cache

The cache is divided into multiple fixed-size blocks, also known as cache lines. If there are 2^n lines. Then there are 2^m partitions in the main memory. Each partition is 2^n lines.

Tag have m bit and index have n bit.



In a direct-mapped cache, a conflict occurs when multiple memory blocks from the main memory map to the same specific cache slot or line.

Set-associative Cache

IDEA: use cache index to select **set** of blocks. Each set is like a fully associative cache.

2 way = 2 blocks. 4 way = 4 blocks.

Use the index to search in sets. It is harder to conflict. Needs more data than the # of ways to conflict.

Reuse distance

$$\text{reuse distance} = \#\text{ of unique other blocks at this point}$$

For example, reuse distance for A at "ABBCBCCDBBEDBBBA" is 4 since there are 4 **unique** blocks before it.

Replacement policy

Algorithm used to decide which vlock to evict.

Some ideas for choosing **replacement victim**

- random (Better than you would expect!)
  - Select victim uniformly at random
  - Pros: do not need to remeber anything!
- least frequently used (LFU)
- least recently used (LRU)
  - Timestamp per cache block
  - Mark the ones to '1' when re-referenced. If all get '1', flip them all to '0'.
  - When choosing ont to evict, choose the ones with '0'.

The 3 Cs (3 types of misses)

- Cold start misses
  - first time a block is accessed
  - increasing size or associativity does not helps
  - can increase block size
- Capacity misses
  - access would not be a miss if cache were infinite
- Conflict misses
  - set size ¡ number of accesses that map to given set
  - access would not be a miss if cache were fully associative

Cache Performance Impact

Miss-oriented Approach to Memory Access:

$$CPUtime = IC \times \left(CPI_{Execution} + \frac{MemAccess}{Inst} \times MissRate \times MissPenalty \right) \times CycleTime$$

$$CPUtime = IC \times \left(CPI_{Execution} + \frac{MemMisses}{Inst} \times MissPenalty \right) \times CycleTime$$

- $CPI_{Execution}$ includes ALU and Memory instructions

Separating out Memory component entirely

- AMAT = Average Memory Access Time
- CPI_{ALUOps} does not include memory instructions

$$CPUtime = IC \times \left(\frac{AluOps}{Inst} \times CPI_{AluOps} + \frac{MemAccess}{Inst} \times AMAT \right) \times CycleTime$$

$$\begin{aligned} AMAT &= HitTime + MissRate \times MissPenalty \\ &= (HitTime_{Inst} + MissRate_{Inst} \times MissPenalty_{Inst}) + \\ &\quad (HitTime_{Data} + MissRate_{Data} \times MissPenalty_{Data}) \end{aligned}$$

Victim cache

Victim cache is a small fully associative cache that stores **recently evicted** lines.

- On evict, place in victim cache
- If hit in victim cache, place back to normal cache
- Managied similarly to normal cache. E.g., LRU
- Adds some extra associativity for some busy lines.

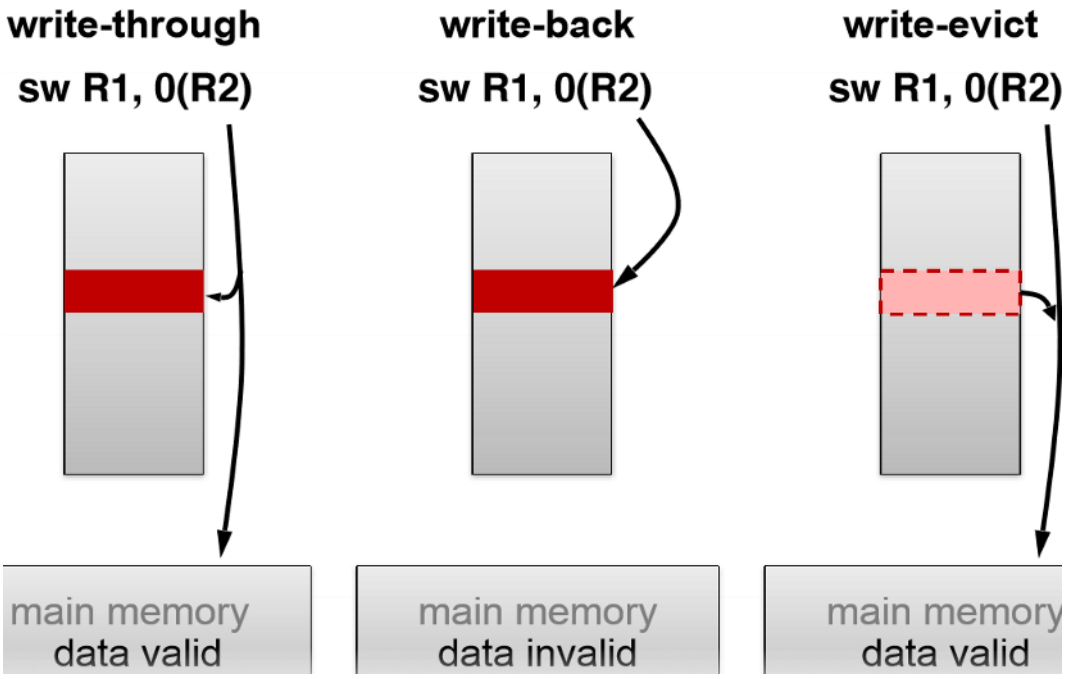
Types of writes that hit in the cache

- Write-Evict:
  - The data is directly written into the memory, then the corresponding cache line would turn valid bit to 0 so that it could be evicted later.

  - Might be good for multicore.

- Write Through:
  - Write the new data into the memory and the corresponding cache line.
  - Will be good if need multiple visits of the same cache line.

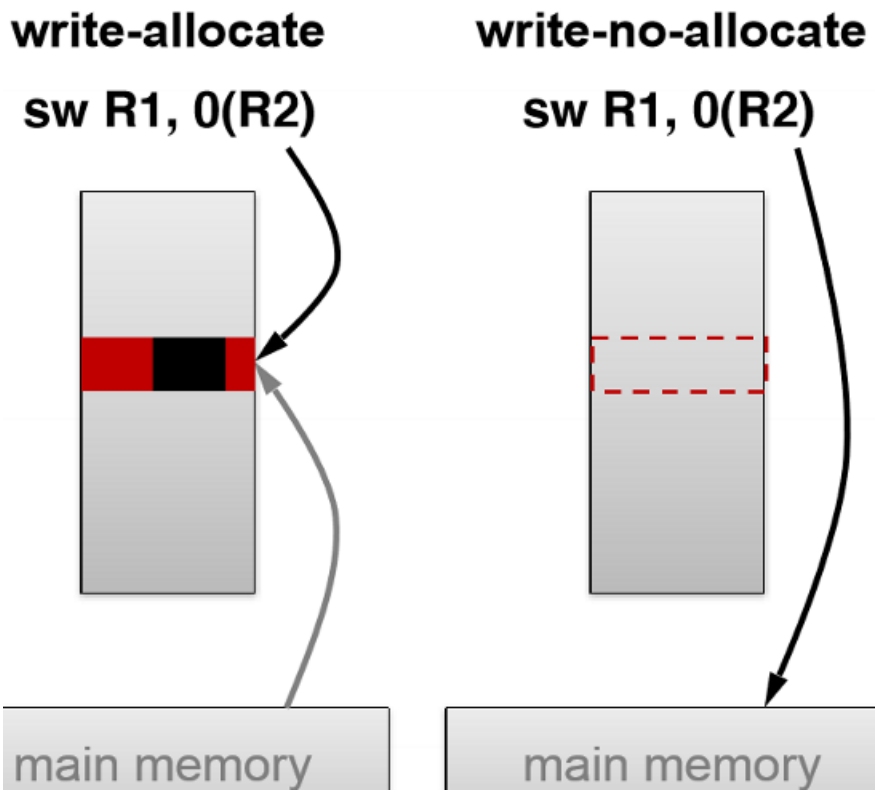
- Write back:
  - Only writes to the cache, memory only gets the data when evicted from the cache
  - Since only writing to cache, it is much faster.
  - However, for multicore, other cores might need to fetch inside a different core's cache to find the right data, which could be even slower.



Types of writes that misses in the cache

- Write-allocate:
  - The corresponding memory block is loaded into the cache, then we write to the cache. So read then write.
  - Apporopriate when subsequent accesses to the same block are likely (exploits temporal locality)
  - Can result in unnecessary reads for write operataions if the entire block is not going to be used.

- Write-no-allocate
  - Write is performed directly to the main memory without loading to the cache. So no read.
  - Reduces cache pollution by avoiding unnecessary reads.
  - More suitable for write intensive workloads where reads of recently written data are less likely.
  - Not able to take advantage of spatial locality
  - Inefficient use of cache when read is following write

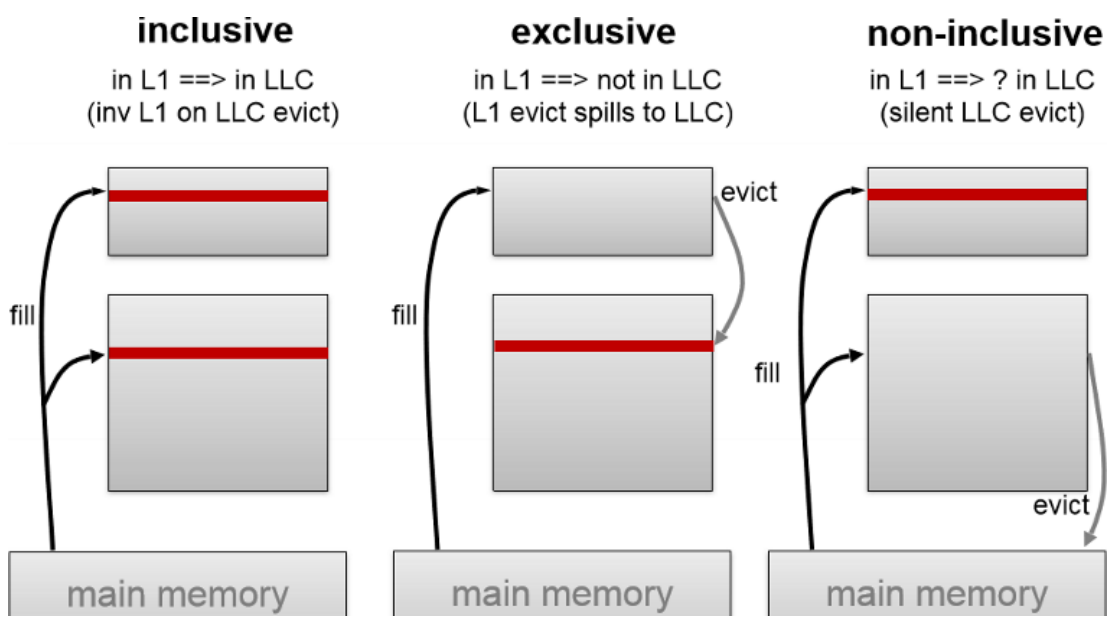


Evictions in a cache hierarchy

- inclusive
  - Lower-level caches contain a superset of the data present in higher level caches.
  - E.g., if present in L1, present in L2
  - Increased storage requiremnts since stored in multiple cache.
  - Fetching is simple since only need to check one level.
- exclusive

- Each cache level contains a disjoint set of data. No duplicate in different levels.
- No redundency so saves storege.
- May result in higher miss rates especially if data frequently migrates between cache levels.
- Need to search all cache to find the data.

- non-inclusive
  - Non-inclusive caches are also known as non-inclusive, non-exclusive or sometimes as set-associative caches.
  - It is unknown if each level has unique or shared data.
  - Compromise between storage efficiency and simplicity in coherence management.



Read and Write

When do you cause memory wirte?

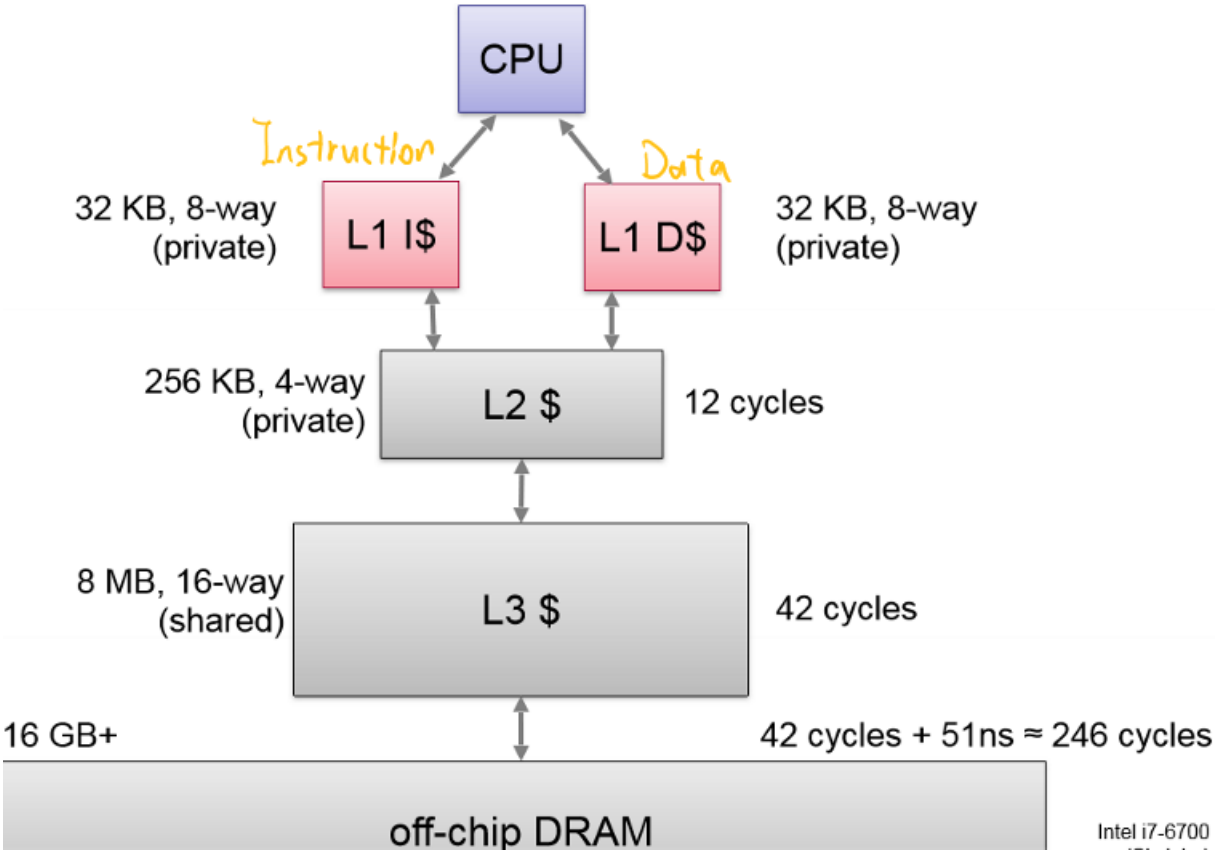
load: reading data from memory

store: writing data to memory

When you load and there is a dirty/modified line, then you write that line to the memory and then load it.

When you store, we evict a cache line which can welcom a memory read.

L1 Separating instruction and data



We separate instruction and data in L1 because:

- Reduce latency in instructions so that there is no instruction load stalls.
  - Instruction stall, also known as pipeline stall or pipleline bubble.
  - Wastes CPU cycles.
  - Instruction pipelines are designed to process multiple instructions simultaneously by breaking down the instruction execution into stages (fetch, decode, excute, etc.). Stalls disrupts the smooth flow of instructions through these stages, leading to a reduction in the overall instruction throughput. This directly impacts the performance of the processor.
- Avoid friendly fire: if togeather, data accesses might evict program's instructions.

Lookup-free cache. MSHR

MSHR = Miss Status Handling Register.

MSHR is a data structure or a set of registers within a cache controller that keeps track of the status of cache misses. When a processor issues a load or store instruction

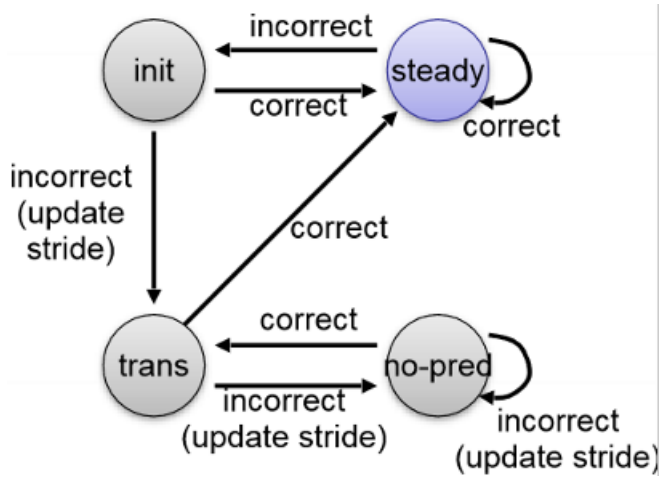
and encounters a cache miss, the MSHR helps manage the process of fetching the missing data from the next level of the memory hierarchy (usually main memory) and updating the cache.

- MSHR keeps track of the cache misses that are currently in progress. Each entry in the MSHR corresponds to a pending cache miss. This allows the cache controller to manage multiple concurrent misses effciently
- MSHR stores information about thememory address that caused the cache miss. This information is used to identify and match incoming data when it returns from main memory.
- MSHR also holds a *Data Buffering* to store the incoming data associated with a cache miss. It holds the data till it can be written to the cache.
- This way, it groups all data going to 1 line and send only 1 request?

Prefetching

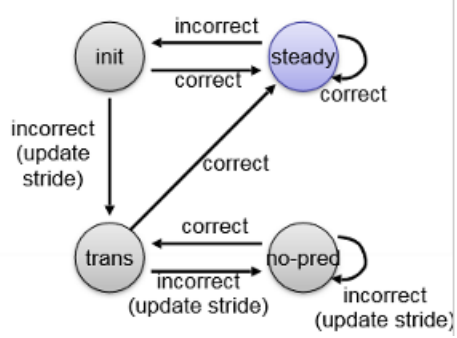
Prefetching is a technique used in computer systems to improve memory access performance by fetching data into a cache or main memory before it is actually needed by the processor. The goal of prefetching is to reduce the latency associated with memory access, especially in situations where the processor is likely to encounter cache misses.

- Use patterns usually to observe next memory access
- Hardware Prefetching: Modern processors often have dedicated hardware units that analyze memory access patterns and automatically issue prefetch requests to fetch data into the cache.
- Software Prefetching: programmers or compilers can also insert prefetch instructions or hints in the code to suggest which data should be prefetched. However, this requires a good understanding of the program's memory access patterns.
- Sequential Prefetching: Predict that the processor will access contiguous memory locations. Effective for loops or linear data access patterns.
  - Stream buffer: buffer sequential access stream. maybe several buffers for multiple streams.
  - if hit in stream buffer, move the whole buffer to cache.
  - However, if next fetch jumps, then it does not work.
- Stride Prefetching: Predict the stride (the constant difference between consecutive memory addresses) and perfetching data at regular intervals. Beneficial for non-contiguous but predictable memory access patterns.

- Track previous address, stride, correctness.
- if predictions confirmed, keep prefetching:
- An FSM approach:
- 

- An example flow chart:
- A stride prefetcher has an FSM and is initialized as shown below
The observed access sequence is
4, 8, 12, 16, 10, 20, 30, 40
How many will be prefetched? How many are correct/incorrect?

prev	stride	state	prefetch
0	0	init	—
4	4	trans	—
8	4	**steady**	12
12	4	**steady**	16
16	4	**steady**	20
10	4	init	—
20	10	trans	—
30	10	**steady**	40



- Stream Prefetching: Similar but more addptive to Stride. It dynamically adjusts the prefetch distance based on the observed memory access behaviour.
- Prefetching pros and cons:
  - Reduced memory latency.
  - Imrove memory subsystem throughput, allowing the processr to countinue executing instructions without waiting for memory accesses.

- Enhance cache uitilization: Prefetching helps make more efficient use of the cache space by populating it with data that is likely to be accessed soon.
- However not always Beneficial, incorrect prefetching decisions can lead to wasted bandwidth can cache pollution. If miss rate is too high, CPU will turn predictor off.

Improve Cache Articles:

1. Adaptive Insertion Policies for High-Performance Caching ISCA 2007: This is present in older CPUs:`https://dl.acm.org/doi/10.1145/1250662.1250709`

2. Tree-Based P-LRU (pseudo-LRU) Replacement Policy: This is present in older embedded CPUs. Please read Chapter 3, Section 4 of this document from ARM.

The Wikipedia page gives you a high-level overview:

`https://en.wikipedia.org/wiki/Pseudo-LRU`

3. SRRiP and DRRiP Policies ISCA 2010: This is present in modern CPUs –¿ `https://dl.acm.org/doi/10.1145/1815961.1815971`

4. Signature-based hit prediction MICRO 2011: This is an academic work –¿ `https://dl.acm.org/doi/10.1145/2155620.2155671`

- Victim selection policy: Decides which line gets evicted for storing an incoming line.
- Insertion policy: Decides where in the replacement list the incoming line is placed.

1. LRU: Least recently used. In an LRU cache, data is stored in a limited-sized cache memory, which can hold a fixed number of items.

   Each time an item is accessed or used, it is marked as the most recently used item. This timestamp or marker is typically recoded in a data structure like a linked list or an ordered array.

   When the cache reches its capacity limit and a new item needs to be added, the LRU algorithm identifies and evicts the least recently used item from the cache.

   LRU is designed to optimize cache usage for scenarios where some items are accessed much more frequently than others.

   One noteable drawback is that it requires maintaining a record of the access order, which can be memory-intensive.

   Also, it might not be good when access patterns change suddenly or there are hot items that are very frequently accessed, which can cause frequent cache evictions.

2. LIP: LRU Insertion policy. Instead of placing all incoming lines into MRU(Most recently used), they are all put into LRU and only prompted to MRU if they get referenced while in the LRU position.

LIP prevents thrashing for workloads that reuse a working set greater than the available cache size.

However, LIP may retain the lines in the non-LRU position of the recency stack even if they cease to be re-referenced. Since LIP does not have an aging mechansim, it may not respond to changes in the working set of the given application.

3. BIP: Bimodal Insertion Policy. Similar to LIP except that it infrequently (with a low probability) places some incoming lines into the MRU position.

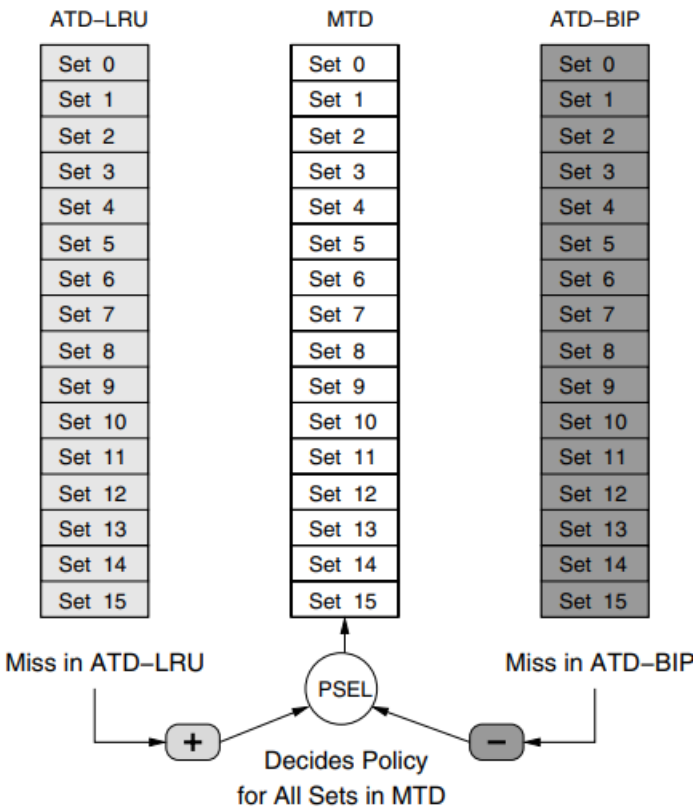
   BIP is regulated regulated by a parameter, bimodal throttle parameter ϵ, which controls the percentage of incoming lines that are placed in the MRU position.

   Both LRU and LIP can be seen as a special case of BIP with $\epsilon = 1, 0$ respectively.

4. Dynamic Insertion Policy BIP(DIP) Dynamically estimates the number of misses incurred by the two competing insertion policies (BIP and LRU). And select the policy that incurs the fewest misses.

   Simiplest way is to implement LRU and BIP in two extra tag directories and keep track of which of the two policies is doing better. The main tage directory of the cache can then use the policy that incurs the fewest misses.

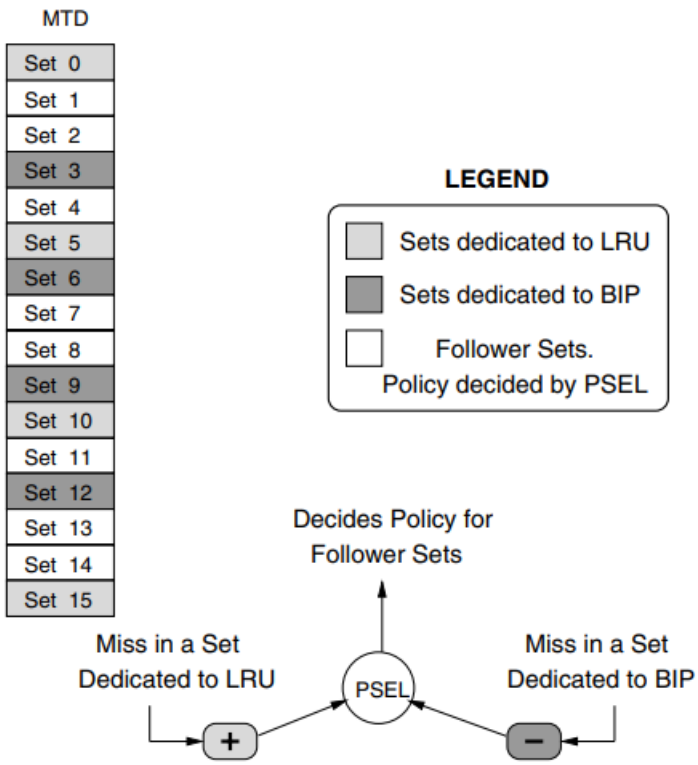
   Figure below demonstrates the working of DIP. Let MTD be the main tag directory of the cahe, The two competing policies LRU and BIP are each implemented in a separate Auxiliary Tag Directory (ATD). Both ATD-LRU and ATD-BIP have the same associativity as the MTD. The access stream visible to MTD is also applied to both of them. A saturating counter, which we call *Policy Selector* (PSEL), keeps track of which of the two incurs few misses. All operations on PSEL are done using saturating arithmetic. A miss in LRU increments PSEL and a miss in BIP decrements PSEL. The most significant Bit (MSB) of OSEL is an indicator of which of the two policies incurs fewer misses. If MSB od PSEL is 1, MTD uses BIP, other wise LRU.



However, it does require two more ATD which can be troublesome. So instead, we introduce **Set Dueling:**

Set Dueling dedicate few sets of the cache to each of the two competing policies. The reset sets are follower sets that uses policy that incurs fewer misses on the deicated sets.

This kind of DIP is called *DIP-SD*



A miss in LRU sets increments PSEL whereas a miss in BIP decrements PSEL. If the MSB od PSEL is 0, the follower sets use the LRU policy. Otherwise the follower use BIP.

4 Virtual Memory

Brife intro to why virtual memory:

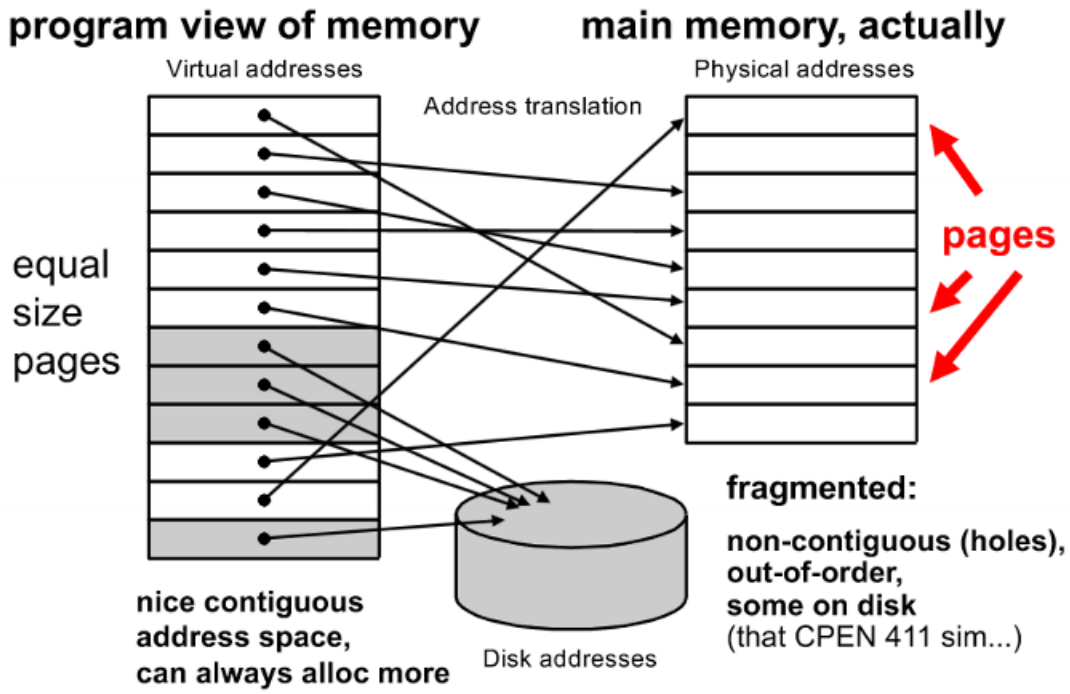
Problem: Operating system needs to multitask (time-multiplex)

- But all programs assume address space starts at 0
- Enough physical RAM to run one program but not enough to fit all of them at once.

Solution: separate address space per program

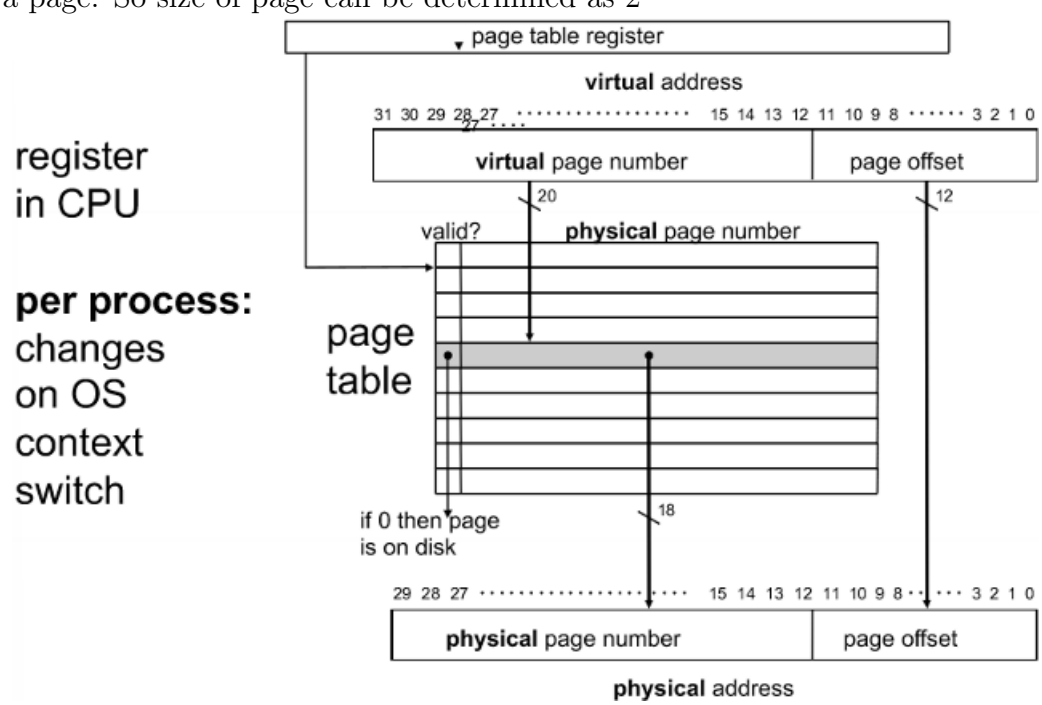
- Each program has separate address space ID (ASID)
- Program executes using virtual addresses (VA), so that the program thinks it has all of memory.
- CPU and OS translate to physical address (PA)
- Same VA with different ASID would map to different physical address.

Paged virtual memory



Divide address into pages. Map used ones into physical address in memory. Leave the rest in Disk.

The physical address is translated from VA by OS. Offset is the byte address of a byte in a page. So size of page can be determined as $2^{\text{offset bit}}$



Page table register is used to map. Valid bit in it indicate if it is on the disk on in the

memory. if (valid == 0) on disk.

Address of the virtual page number in the page table register holds the physical address.

Size of the table depends on the bit number of virtual page number since the page table sohuld cover all of the virtual pages.

For example, for a n bit virtual address and m bit page offset, virtual page number is (n-m) bit. So number of entries for the table is 2^{n-m}

For programs that use virtual memory, address translation is required for every data mem access and every instruction fetch since it does not have the actual address of data.

Efficiency of page table

Where to put the page table?

- Option 1: on-chip near the CPU (like the L1 Cache)
  - pros: very fast access
  - cons: 1 M entries size >> any reasonable L1 cache
- Option 2: off-chip in main memory
  - pros: size might not be a problem.
  - cons: access is very slow (We need 2 accesss for each instruction)
  - cons: per-process tables is still lots of memory.

Observations in real programs:

- Each process needs own page table. 100s-1000s processes could easily be active.
- Page tables are sparse: few programs use the entire address space.

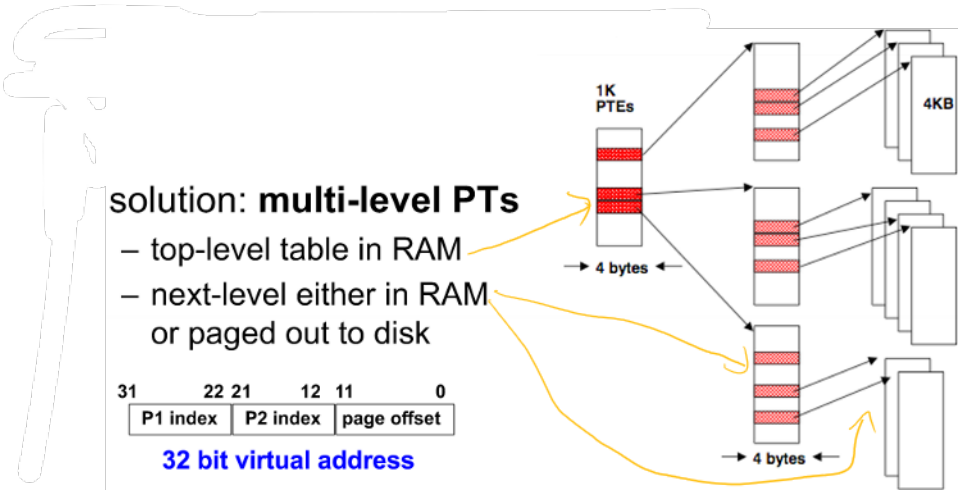
Solution: Multi-level page tables (a.k.a hierachical page tables). Instead of using a single, flat table to map virtual addresses to physical addresses, a multi-level page table divides the task into multiple levels of tables, forming a hirerarchy.

- Depending on the number of levels, higher-order bits are used as an index into the first-level page table.
- The entry in the first-level page table points to the second-level page table.
- The lower-order bits are used as an index into the second-level page table, which contains the final page frame information.

Benefits:

- Space Efficiency: reduce the amount of memory needed for the page table, especially when dealing with sparse address spaces where not all address space is in use.

- Flexibility: Different levels of the page table hierarchy can have varying sizes, allowing for flexibility in optimizing the memory usage.



Translation-related exceptions

- Page faults:
  - page is accessible, but it is sitting on disk. It happenes all the time!
  - this is not a program error!
  - resume instruction execution once page in RAM.
- Segmentation fault/ general protection fault
  - page is not user-accessible
  - page is R/O (read-only) but it tries to write/execute.
  - program terminated (unless caught)
  - Meltdown avoided this by misspeculating.

Translation Lookaside Buffer (TLB)

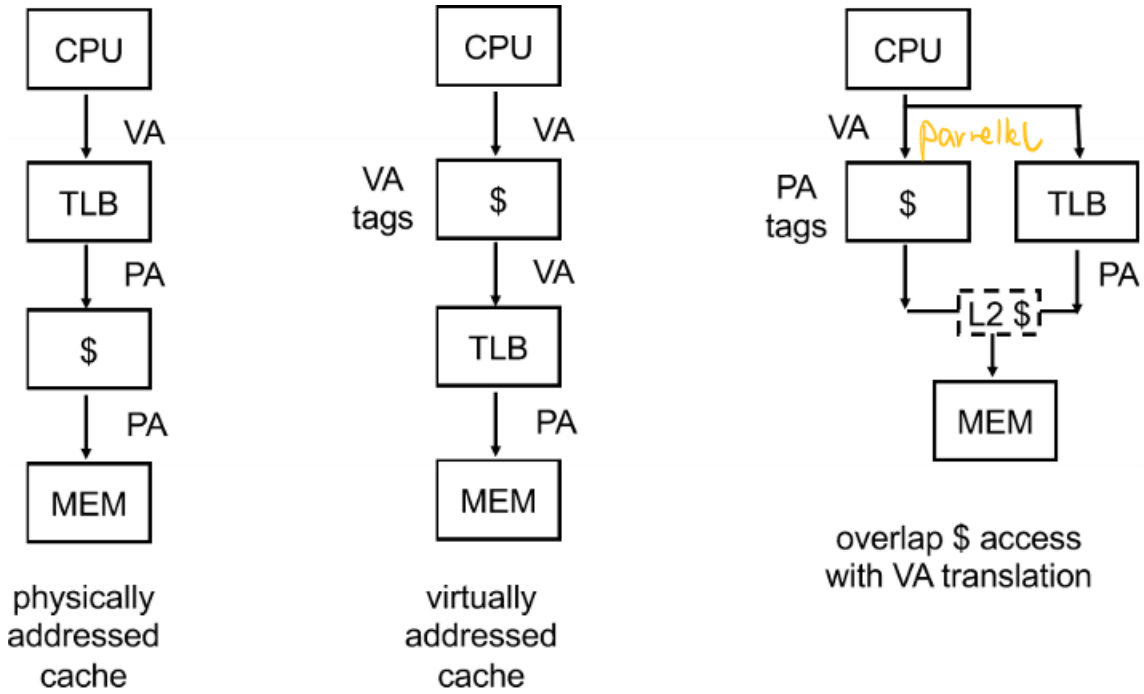
VA to PA translation is needed for every memory access. Lookup in big multi-level page table is really slow.

We use TLB, which is a small cache for most frequently used virtual page number (VPN) to physical page number (PPN) lookups.

- If hit, saves a lot of cycles.
- If miss, engage page table walker to look for it.

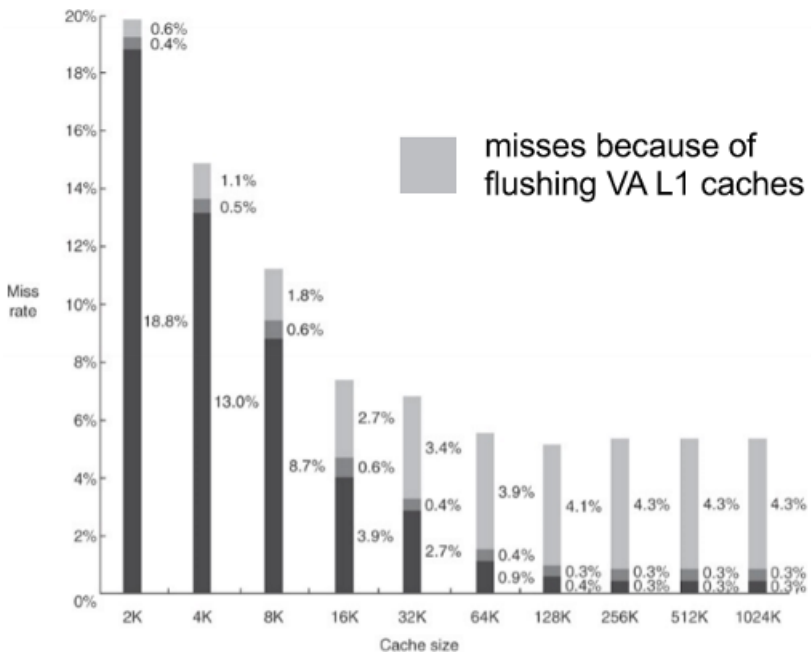
Upon page fault, we evict old page and bring in the new page. When switching to a different program, we need to flush TLB since it is in a different virtual address space.

Types of address translation

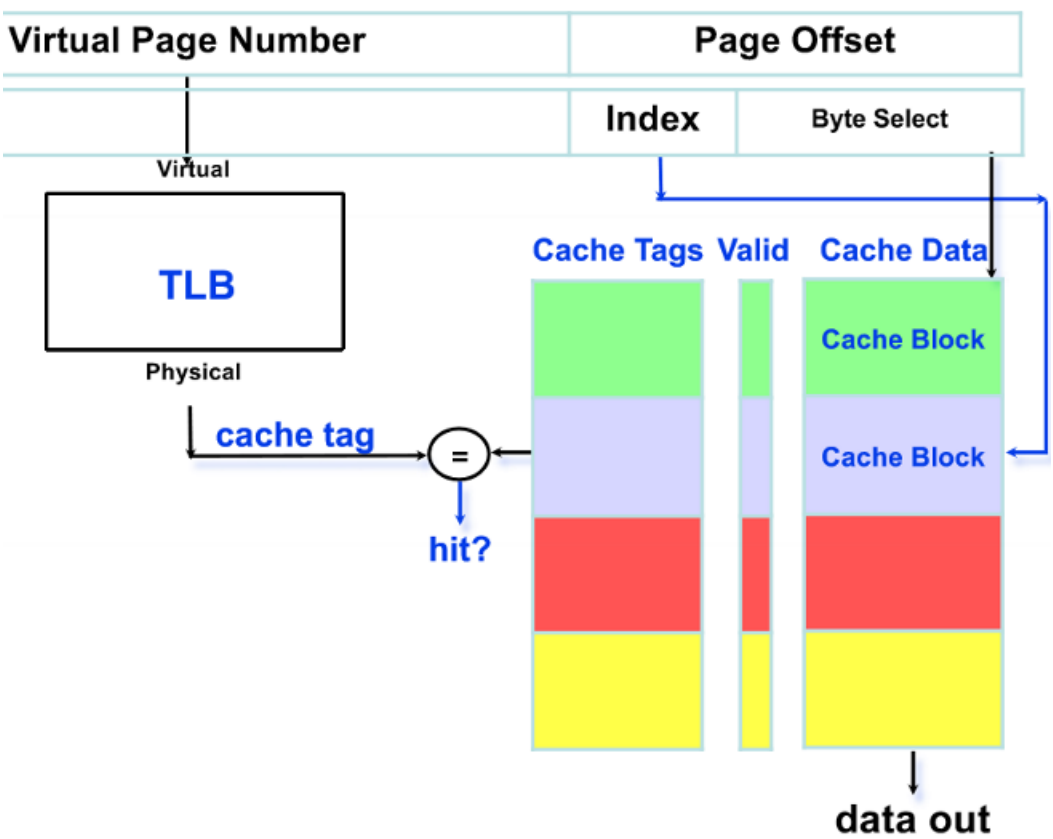


- Physically addressed cache: If hit in TLB, directly translate VA into PA and access it. Else, look into PTE mem.
- Virtually addressed cache: Cache only contain VA of one program only. Then we can go from cache, if miss then TLB then mem. Otherwise PTE.
- Overlap is obviously the fastest. But the structure needs some special arragement.

Increasing the size of L1 cache would also increase the misses due to flushing VA L1 caches:



Overlapping TLB and L1 cache accesses



Example problem:

Suppose pages are 4KB. D cache is 2-way set-associative maximum cache size?
Page = 4kb = $2^{12} \implies$ set = 6bit = 2^6 and byte = 6 bit = 2^6.
Set associativity is 2: size = $set \times associativity \times byte = 2^6 \times 2 \times 2^6 = 2^{13} = 8$KB.

VA to PA and synonym problem

Problem: some address bits change when mapping VA to PA because page is larger than the page offset compacity. For example, index of 7 bits are the offset of 6 bits but your page offset is 12 bits only. Possible solutions:

- bigger page sizes. (Increase the page offset to be 13)
- more assoviativity. (Allow a 2 way associativity)
- Make sure the VA keeps the extra index bit the same as before. So maybe there are only half of VA that it is allowed to get in. (Hard to do when there are more bits needed to keep the same)
- synonym problem

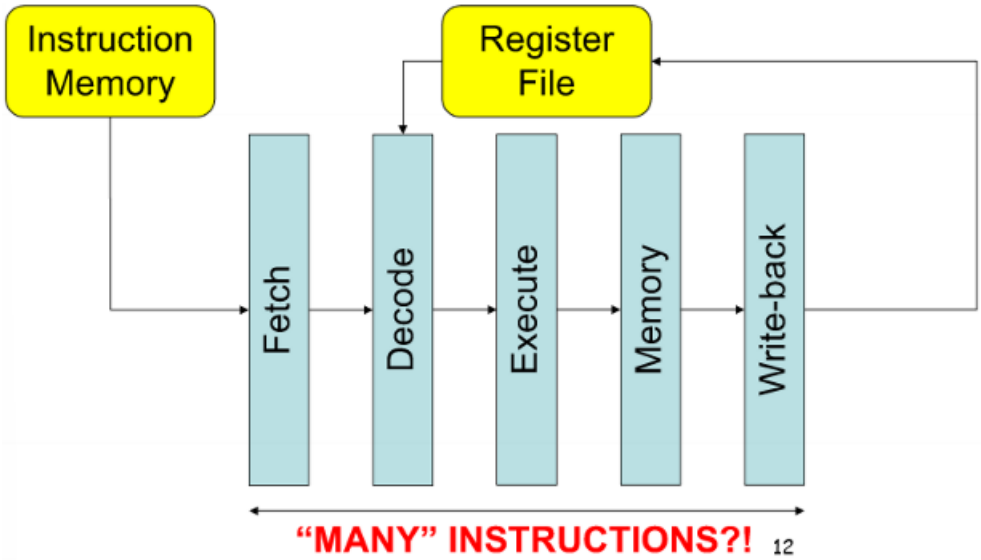
Sunonym problem example:
We have a 64KB direct-mapped cache and 16-byte blocks.

- total length is 16 since 2^{16}bit=64KB
- block offset is 4bits, so addr[3:0]
- The reset is cache index, addr[15:4]

If we only have a 4KB page, then each address of page correspond to $\frac{64}{4} = 16$ other cache. So 16 synonym locations for every physical address.

5 Pipelines

Let's look at a simplefied pipelining structure.



Each stage is a pipeline register. We do one thing at one stage.

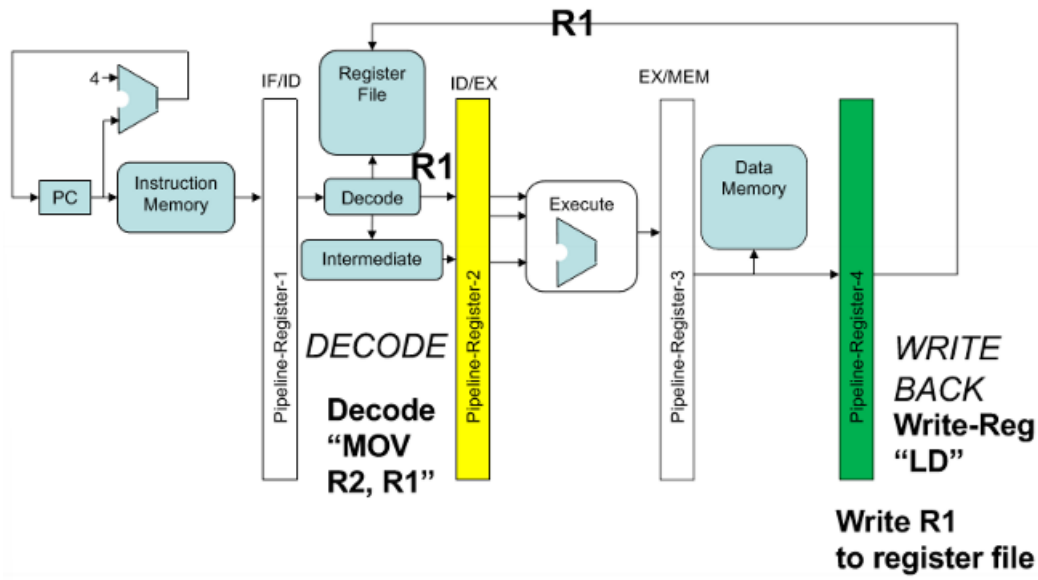
However, flip-flops have latency. So we cannot divide too many stages since the charging latency of flip-flop limits it.

Ideal pipeline:

- Uniform subcomputations: each stage has same delay, achieve by balancing pipeline stages.
- Identiical computations: each computation uses same number of stages.
- Independent computations: start new computation each cycle.

Dependency

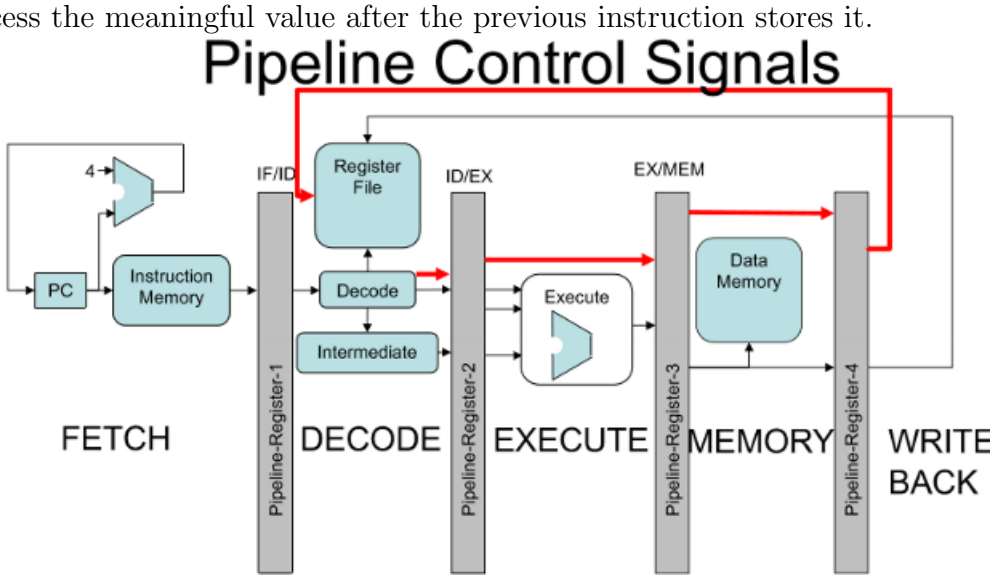
A problematic pipeline:



Take a closer look at LD and MOV instruction

If we are supposed to read the written R1 before decoding it, then the R1 value read before it is "stale value".

So reading the value depends on the previous instructions. Need to make sure you can access the meaningful value after the previous instruction stores it.



We use the signals that goes along with the instructions. The signal can stall the piepline so that we can make sure the value is meaningful.

Hazards

	Clock Number								
	1	2	3	4	5	6	7	8	9
ADD R1, R2, R3	IF	ID	EX	MEM	WB				
SUB R4, R1, R5		IF	ID	EX	MEM	WB			
AND R6, R1, R7			IF	ID	EX	MEM	WB		
OR R8, R1, R9				IF	ID	EX	MEM	WB	
XOR R10, R1, R11					IF	ID	EX	MEM	WB

- R1 is the destination of the ADD instruction.
- R1 is also the source of the rest of the instructions.
- R1 introduces dependencies which causes **Hazards**
- In **Cycle 3 and 4**, we need the value of R1, but R1 is updated in **Cycle 5**!
- Everything at or after cycle 5 is ok.

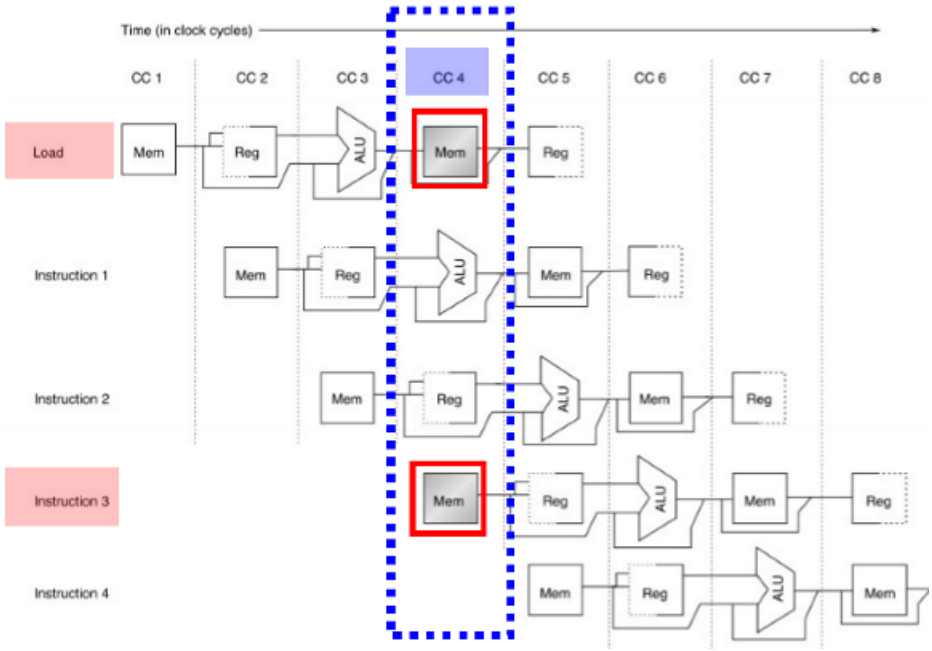
There are three types of hazards

- Structural Hazards
- Data Hazards
- Control Hazards

Structural Hazards

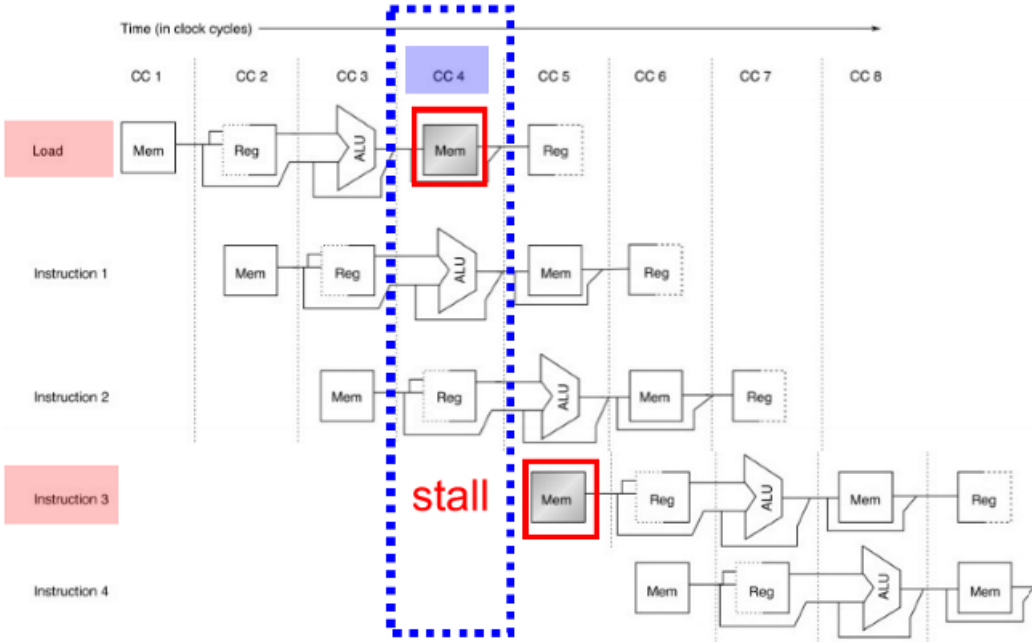
Caused by resource conflicts:

- Two or more instructions want to use the same hardware at the same time.
- E.g., fetch instruction and write to mem at the same time, but you can excute one instruction at a time.
- One solution is to stall it and wait for the earlier instruction to finish using resource.
- The other solution is to add more hardware so that multiple instructions can be executed at the same time.



Example:

So at this example, both instruction want to access the memory at the same time. Solution is to stall the later instructions.

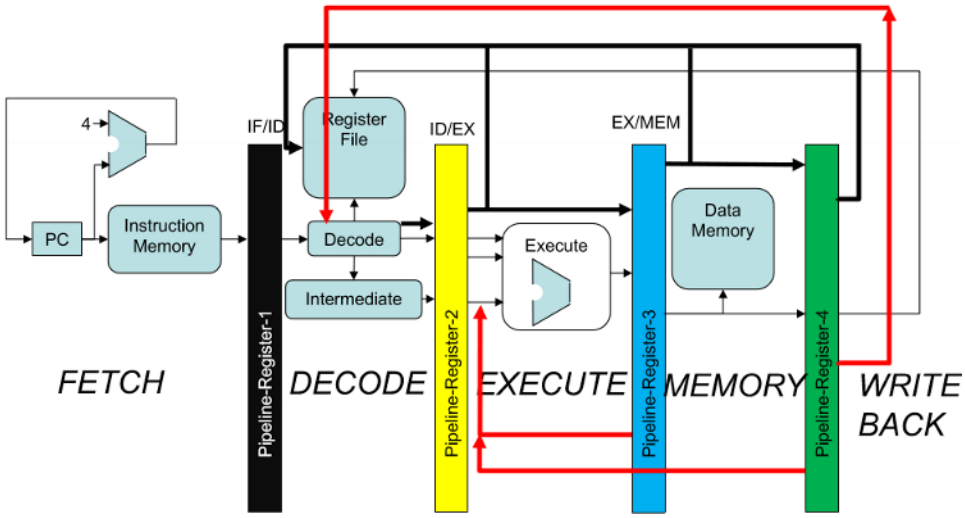


Data Hazards

1. Read after write (RAW) hazards: Note the data dependencies of the following instructions:

```
ADD  R1, R2, R3
SUB  R4, R1, R5
AND  R6, R1, R7
OR   R8, R1, R9
XOR  R10. R1. R11
```

Solution to it is **Forwarding**: Sending the data to the next execute stage as it is in CPU before write back.



By adding a mux for all the next stages so that it is possible for it to forward the data into the next instruction ALU. These muxes are called **Forwarding Mux**

2. Write after write (WAW) hazards:

   An example is "ADD R1, R2, R3" then "SUB R1, R2, R4".

   For this case we can simply ignore the first line since it will be overwritten anyways, which does not cause any problem.

3. Write after read (WAR) hazards:

   An example is "ADD R1, R2, R3" then "SUB R2, R4, R5".

   Just use a temprorary register to keep the value you need to write to R2 or change to a different register selected by the CPU. Not a big deal.

Control Hazards

Control Hazards are due to branches. Branch may or may not change program counters (PC). So which instruction should foollow a branch depends upon the result of executing the branch instructions.

Pipeline can only choose one branch to countinue. If not correct, then the execution is wasted.

- Option 1 is to stall it. Wait for you know the branch value before countinue.
- Option 2 is do branch prediction: Branches tend to be "weakly biased" meaning that they are taken and not taken roughly the same numebr of times. So half of the time, we might be right! So now we need a way to correct the behaviour if predicted wrong.

  If correct, we know by the end of the branch instruction, and we are happy, so nothing we need to do.

  If not correct, at the next cycle, we need to fix all the incorrect predictions. Squash incorrectly fetched instructions (discard) and start fetching correct instructions.
- Option 3 is to delay branches:

  Here is some compiler optimization that can be done.

  Place useful instructions after branch instructions if possible.

  - Put instructions that is before the branch that the branch is **not dependent** on. And ut should always improve the performance.
  - If from branch target: Must be OK to execute if the branch is not taken and improves the performance if it is taken.
  - If from fall through: Must be OK to execute if the branch is taken and improves the performance if it is not taken.

Dependencies

Define dependency: Two instructions use the same reigister or memory location but no data passes between them. Dependency is on data location (name) not value (contents at location).

1. True dependency and RAW hazards: A true dependency leads to a 'RAW' hazard if the hardware can perform the read before the write.

	Clock Number					
	1	2	3	4	5	6
DADD x1,x2,x3	F	D	X	M	**W**	
DSUB x4,x1,x3		F	**D**	X	M	W

2. Anti-dependencies and WAR hazards:

```
I: DSUB x4,x1,x3
J: DADD x1,x2,x3
K: XOR  x6,x1,x7
```

   Instructions I and J cannot be reordered, otherwise value will be changed.

   So anti-dependency reuses the names. If anti-dependency causes a hazard in the pipeline, it is called a WAR hazard.

3. Output dependency and WAW hazards:

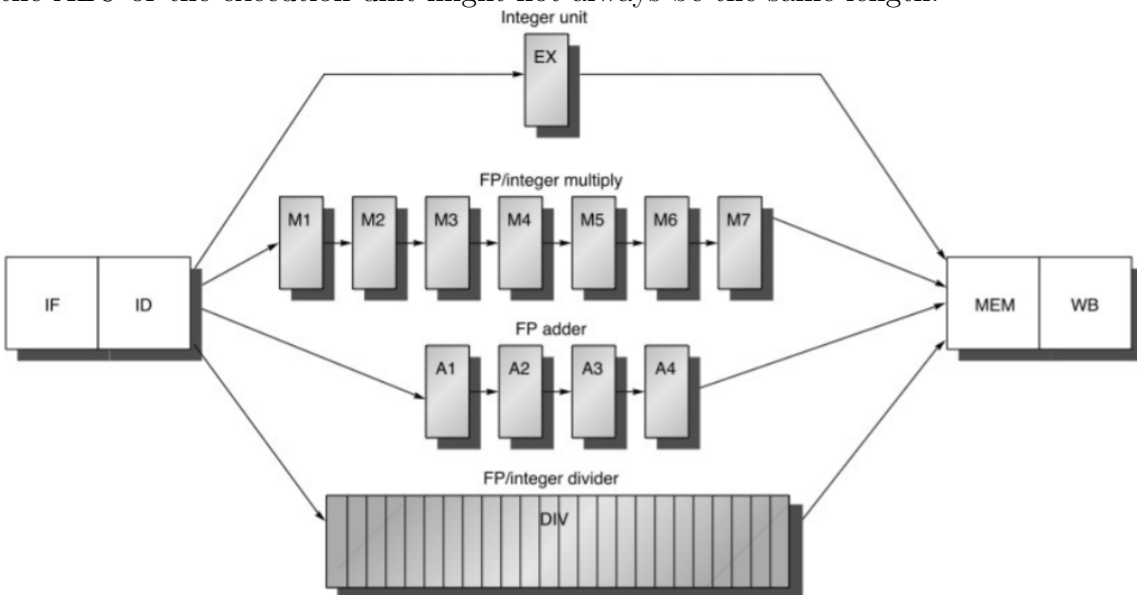
```
I: DSUB x1,x4,x3
J: DADD x1,x2,x3
K: XOR  x6,x1,x7
```

   Instructions I and J cannot be reordered, otherwise value will be changed.

   So it also results from reuse the name. If an output denpendency causes a hazard in the pipeline, it is a WAW hazard.

pipelined function units

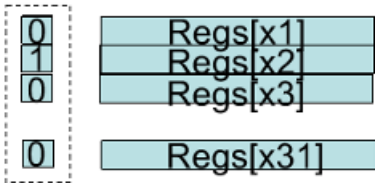
So the ALU or the execution unit might not always be the same length.



So it brings up more chanllenges:

- Longer RAW hazards. (Might need to wait for longer so that you can read the executed value)
- Structural hazard for MEM, WB. (might have multiple request to access them at the same time.)

scoreboard: tracking register accesses



For each reg, track "will be written to by in-flight instruction" info.

- **Set bit** for destination register at end of **decode**.
- **clear bit** when instruction **reaches writeback**.

So if new instruction **reads or will write** a register with the bit set, stall in decode stage.

So for a "in-order" scoreboard execution:

- scoreboard: a bit-array, 1-bit for each GPR(General Purpose Register) if the bit is not set: the register has valid data. If set: the register has stale data. Some in-flight instruction will write them.
- issue in-order: For example: (SB stands for scoreboard)

  RD ← Fn(RS,RT)

  - if (RS.SB()||RT.SB()) RAW.stall();
  - else if (RD.SB()) WAW.stall();
  - else, dispatch to function Fn and set RD.SB to 1;

For compelte out-of-order: Update GPR[RD], clear RD.SB();

Register Renaming

Note that WAW and WAR come from name dependency which can be solved if there are infinate supply of names.

Tomasulo algorithm

Hardware

There is a reservation station in front of each functional unit(FU). FU is a buffer that stores instructions waiting for operands.

- Registers in instructions renamed to tags.
- Destination register gets fresh tage : mapping to reg. alias table (RAT)
- Spirce register get corresponding tags from RAT
- Track data dependencies directly: no WAW or WAR hazards

Results broadcast on **common data bus** (CDB). Each result carries tage of source RS when it is produced. All reigister station (RS) and register file (RF) listen to the CDB for tages they want values for and pickup corresponding value on CDB.

Stages

- issue
  - If have a free RS (no structural hazard), allocate RS entry
  - lookup source operand registers in RF: get either values or tags. (Get value if available, otherwise, need tags since waiting for value)
  - Rename destination register to allocated RS entry tag.
- execute
  - Listen for unready operand tags on the CDB, pickup values.
  - Once all operands ready, commence execution.
- write result
  - Broadcast <dst tag, result> on the CDB
  - Intereseted RSs and RF will see the tag and update value
  - Mark reservation station available

6 In-order commit and multiple issue

The problem rises from the difference of completion and commit:

- Complete: Executiong unit has finished executing the instruction
- Commit: Update the value in the register file

So if we design our CPU that commit and complete is at the same time as how simple Tomasulo algorithm states, RF will be updated at the same time as comletion. However, might need to discard some values if we found some exception in completion. Which means ther might be some that we should not update into the RF.

Why would we flush the piepline? Exceptions!

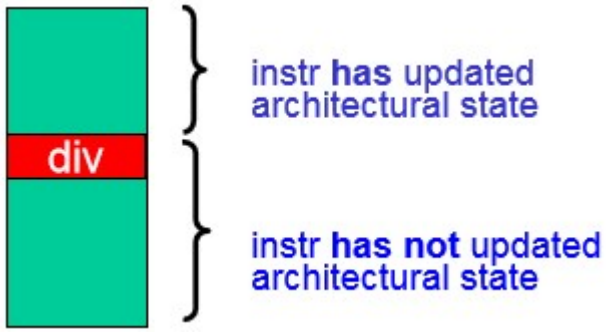
- Mispredicted branch
- External interrput (key press)
- Program fault (div. by 0, segfault)
- Trap (debugger breakpoint)
- OS context switch (timer)

- Page fault (unmapped virtual address)

All above are called exceptions. For each exception that we detected at **execution time,** it requires **transfer of control** to an OS interrupt service routine. Then we may need to **resume the original program** as if nothing happened.

Precise exceptions

If an architectural state is updated by **all instructions before faulting instruction** and **none** after (and including) faulting instrction, then pipeline supports **precise exceptions**. Note that not all architectures do.



Exceptions in an in-order pipeline

When exception strikes:

1. switch instrction fetch to **exception handler**
2. convert instructions after and including "faulting instruction" to **nops** (e.g., turn off write enable control bits)
3. **remember PC** (program counter) of faulting instruction

Dealing with exceptions without stalling

Speculative Execution

Goal of speculative execution is to support excetion of instructions fetched following a branch prediction <u>before</u> we know if the prediction is correct.

However, the problem is that instructions after the branch should not update register file before comfirming that the branch is correct. Make sure control/branch hazard is resolved before proceeding.

Speculative execution describes the capacity of the way to **cancel mispredicted path**.

Completion and commit

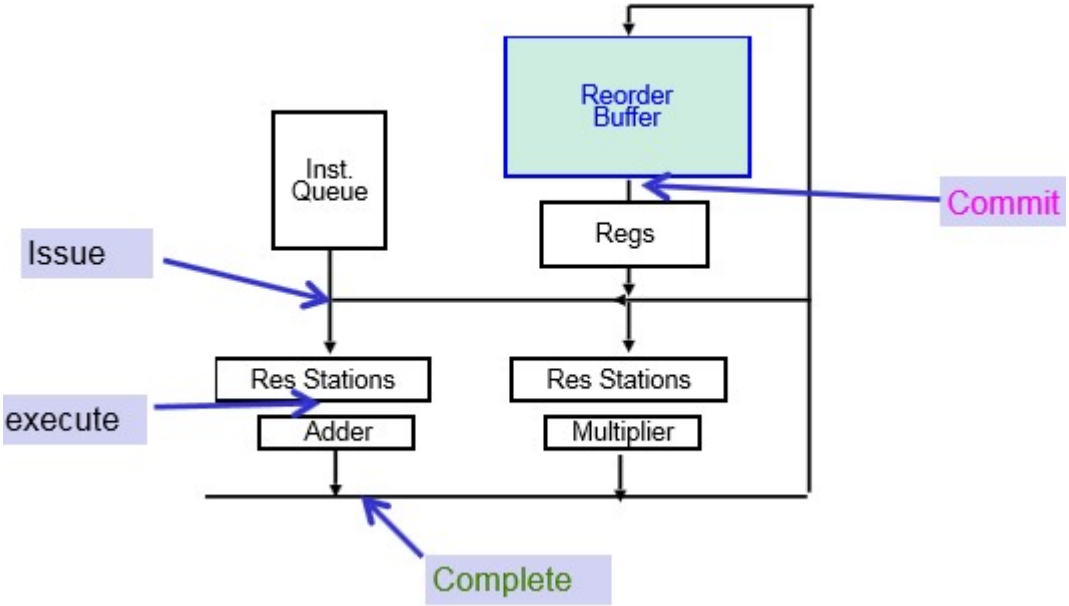
- Insight: problem arises on writing to RF, not finishing the computation early.
- Idea: compute and buffer the result but **delay the RF write** until prior instructions guaranteed to retire (not speculative, no exceptions)

Now we can split writeback into **completion and commit**.

- Completion = buffer computed result
- Commit = write result to RF

The recorder Buffer (ROB)

ROB is a process between completion and commit:



- sores instructions in **program order** (a FIFO)
- **buffers** computed results before RF write
- commits in **program order**

ROB tracks execution status for each instrction

busy?	sent to FU?	finished?	PC	dst reg	speculative?	valid?	*value*

Depending on if there is a branch it is waiting for, issue stage allocates RS and ROB entry may or may not:

- use ROB entry to hold computed value
- use ROB entry ID as renaming tag

Superscalar execution

Pipeline CPI = Ideal CPI + Structural stalls + Data hazard stalls + Control hazard stalls.

We know how to remove/reduce stalls: pipelining, forwarnding, outoforder execution, speculative execution...

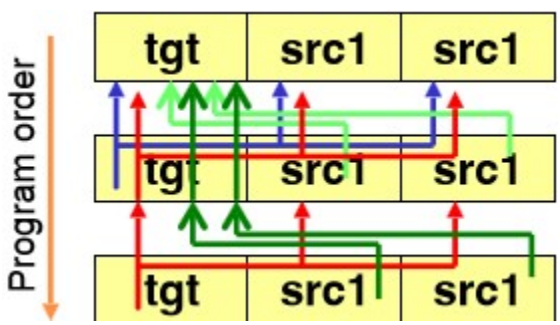
So we also want to reduce ideal CPI:

- ideal CPI limited by issue rate: 1 instruction per cycle.
- what if we allow more instructions to be executed per cycle?

Stactically Scheduled Superscalar

Issue up to N instructions per cycle. At issue:

- check each instrction for hazards with coissued instructions that are **earlier in program order**
- also agianst all earlier **instrctions still in execution**



above: comparisons to avoid RAW, WAR, and WAW hazards in a 3wide inorder superscalar.

Red and blue checks for WAW and WAR. Green checks for RAW.

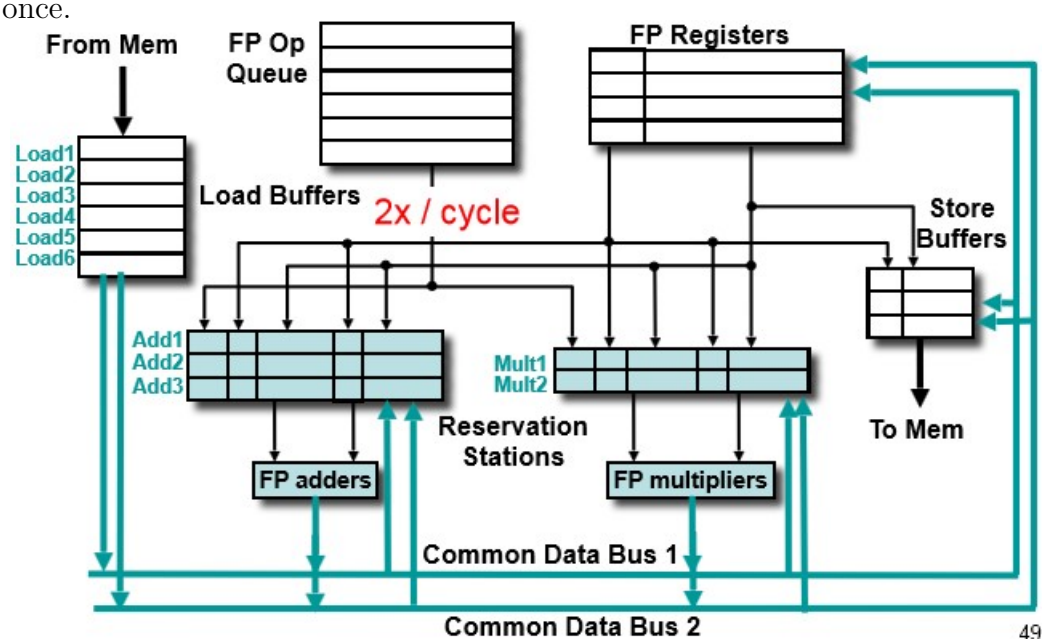
2Issue Inorder Superscalar

	Clock Number							
	1	2	3	4	5	6	7	8
integer instruction	IF	ID	EX	MEM	WB			
FP instruction	IF	ID	EX	EX	EX	WB		
integer instruction		IF	ID	EX	MEM	WB		
FP instruction		IF	ID	EX	EX	EX	WB	
integer instruction			IF	ID	EX	MEM	WB	
FP instruction			IF	ID	EX	EX	EX	WB

So here we have a piepline for Integer and FP. There are mutiple issues with Dynamic Scheduling:

- needs an extension of Tomasulo's algorithm
- key challenge: renames, assign RS, and update pipeline control tables for **for mutiple instrcstions**

We can either run this at a higher clock speed or add logic to rename multiple instructions at once.



So here, we have two CDB which can handle two output at once. Then we run the load buffer at two times speed or two assigning per cycle to allow two instructions adding per cycle.

Branch Prediction

When the piepline is really long, there is a high branch miss penalty!